```
private

  def check_guess
    if @guess > @number
      puts "#{@guess} is too high!"
    elsif @guess < @number
      puts "#{@guess} is too low!"
    end
  end

  def announce_result
    if @guess == @number
      puts 'You won!'
    else
      puts "You lost! The number was: #{@number}"
    end
  end
end

# play the game if this file is run directly
GuessingGame.new.play if __FILE__.end_with?($PROGRAM_NAME)
```

Now, run the code with ruby lib/guessing_game.rb and try playing the game. Get a feel for how it works.

Your mission is to get this class under test. Here are some questions you might want to consider:

- What are this class's collaborators, and how can your test supply them?
- What kinds of test doubles would work best here?
- What edge cases do you need to cover in your specs?
- Should any of this class's responsibilities be extracted into a collaborator?

If you get stuck, you can have a peek at the specs and refactored class we wrote for this exercise. They're in the book's source code.[19,20]

Once you have a solution, please consider posting it in the forums.[21] We'd love to see what you came up with.

Happy testing!

19. https://github.com/rspec-3-book/book-code/blob/v1.0/15-using-test-doubles-effectively/solutions/guessing_game/spec/guessing_game_spec.rb
20. https://github.com/rspec-3-book/book-code/blob/v1.0/15-using-test-doubles-effectively/solutions/guessing_game/lib/guessing_game.rb
21. https://forums.pragprog.com/forums/385